

Parcial 3 – modelo de examen

Ejercicio 1 – Practico

Durante la fase crítica de descenso en Marte, la computadora de vuelo de la misión **CONAE-Marte** utiliza un **árbol binario de decisiones** para seleccionar maniobras según condiciones (velocidad, altitud, temperatura externa, etc.).

Para garantizar tiempos de respuesta razonables, este árbol debe estar **balanceado**, es decir:

- La diferencia de alturas entre subárbol izquierdo y derecho en cada nodo es a lo sumo 1
- Ambos subárboles también son balanceados
- El árbol vacío se considera balanceado

Implementá desde cero:

- La función recursiva `esBalanceadoRecursivo(Nodo*)`
- La clase `BST`, que recibe la raíz en su constructor y expone un método `bool esBalanceado()` que retorna `true` si el árbol está balanceado

```
#include <iostream>
#include <map>
#include <vector>
#include <string>
#include <set>
using namespace std;

struct Nodo {
    int valor;
    Nodo* izquierdo;
    Nodo* derecho;

    Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
};

class BST {
private:
    Nodo* raiz;

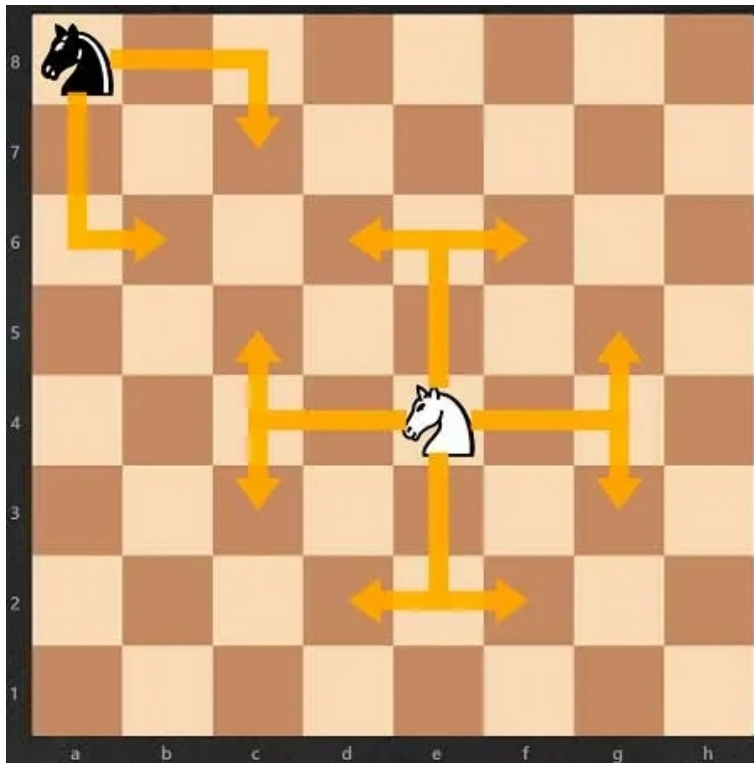
    // Retorna la altura si está balanceado, o -1 si no lo está
    int esBalanceadoRecursivo(Nodo* nodo) {
        // TBD
    }

public:
    // Constructor faltante

    bool esBalanceado() {
        // TBD
    }
};
```

Ejercicio 2 – Practico

En el juego de ajedrez el caballo puede moverse en L (2 casillas en una dirección y 1 casilla en otra dirección transversal)



Desarrollar la función `saltosMinimosCaballo (N, inicio, fin)` , que permita determinar la cantidad de movimientos mínimos del caballo para ir de una posición a otra del tablero.

N : Numero de casillas del tablero

inicio: par (x, y) de la posición de inicio

fin: par (x,y) de la posición final

Pistas:

- Se considera cada posición del tablero como un vertice de un grafo pero de forma implícita .
- Utilizamos una matriz de visitados para controlar si el caballo ya pasó por una posición específica.
- Realizamos un BFS hasta llegar al nodo destino , saltando solo a las posiciones válidas determinadas por los siguientes arrays
- `int dx[] = {-2, -1, 1, 2, 2, 1, -1, -2};`
`int dy[] = {-1, -2, -2, -1, 1, 2, 2, 1};`

```
int saltosMinimosCaballo(int N, pair<int, int> inicio, pair<int, int> fin) {  
    return -1;  
}
```

Pregunta 3 – Teorico

¿Cuál es el principal objetivo del algoritmo de Dijkstra?

Pregunta 3 Respuesta

a.

Encontrar el camino más corto entre todos los pares de nodos en un grafo.

b.

Encontrar el camino más corto desde un nodo fuente único a todos los demás nodos en un grafo con pesos de aristas no negativos.

c.

Determinar el árbol de expansión mínima de un grafo.

d.

Detectar ciclos en un grafo dirigido acíclico (DAG).

Pregunta 4 – Teorico

¿Qué estructura sería mejor para representar la agenda de operaciones quirúrgicas donde siempre debe accederse a la más cercana en el tiempo?

Pregunta 4 Respuesta

a.

Pila

b.

Cola circular

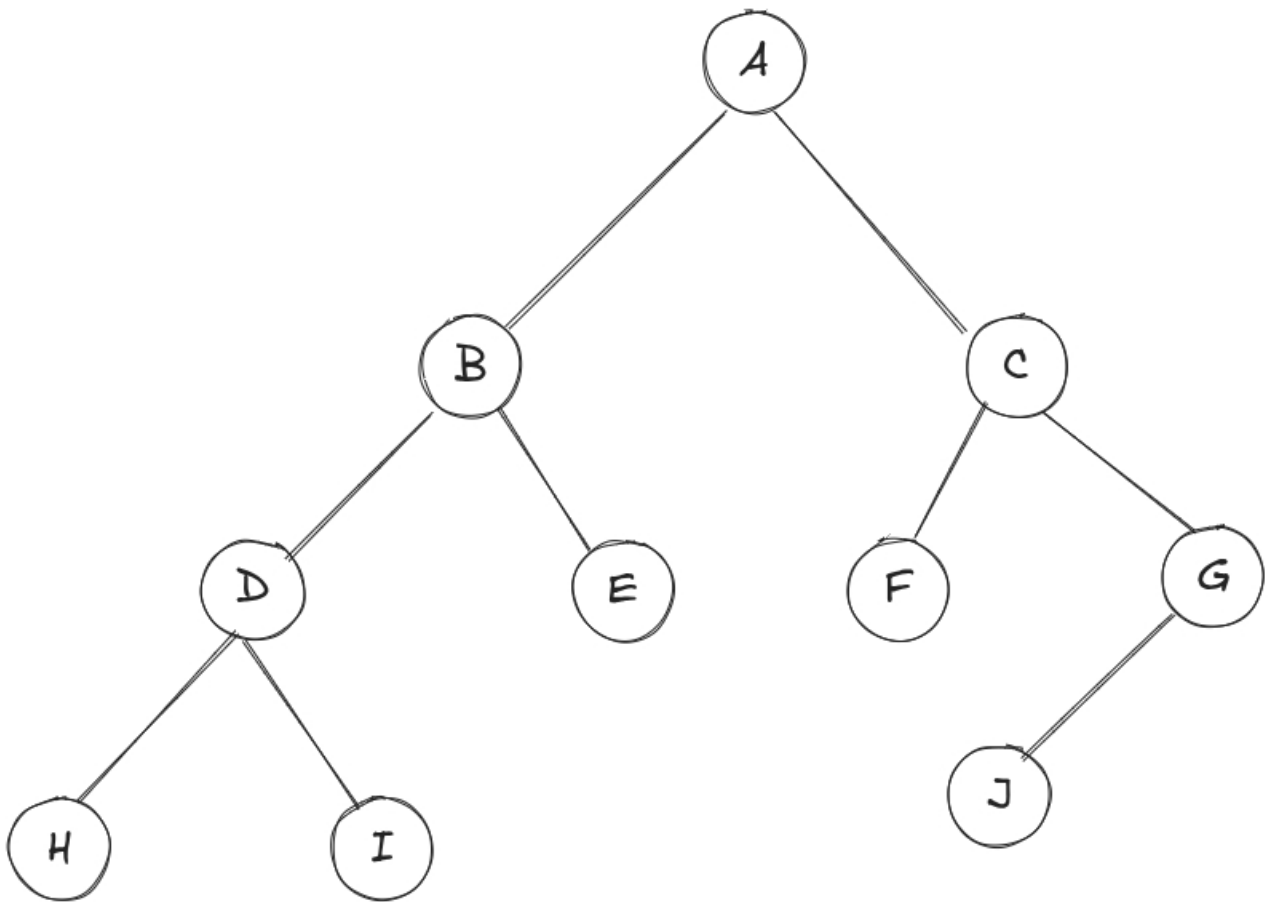
c.

Cola de prioridad

d.

Lista simple

Pregunta 5 – Teorico



El recorrido **inorden**, los nodos recorridos son...

Pregunta 5 Respuesta

a.

[A, B, D, H, I, E, C, F, G, J]

b.

[A, B, D, H, I, E, C, G, F, J]

c.

[H, D, I, B, E, A, F, C, J, G]

d.

[D, H, I, B, E, A, F, C, J, G]